

DBMS LAB – TASK 5 – CSE 3

SQL Aggregate Functions, GROUP BY, HAVING and ORDER BY using the Taxation Database

1. Aim

To understand and apply SQL Aggregate Functions, GROUP BY, HAVING, and ORDER BY to analyze taxation data and generate meaningful summary information.

2. Objective

The objective of this laboratory experiment is to help students understand how SQL can be used to summarize and analyze multiple records instead of retrieving only individual records.

By the end of this laboratory experiment, students should be able to:

- Apply aggregate functions such as **COUNT()**, **SUM()**, **AVG()**, **MAX()**, and **MIN()**.
- Group records using **GROUP BY**.
- Filter grouped results using **HAVING**.
- **Sort results using ORDER BY**.
- Combine aggregate functions with JOIN operations.
- Generate meaningful taxation reports using SQL.

3. Prerequisites (Micro Topics)

Module 1 – SQL Basics

- SELECT
- FROM
- WHERE
- Comparison Operators

- Logical Operators
- ORDER BY

Module 2 – SQL Functions

- String Functions
- Numeric Functions
- Date Functions
- Conversion Functions

Module 3 – SQL JOIN Operations

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- Joining two or more tables
- Table aliases

4. Introduction

In the previous laboratory, students learned how SQL JOIN operations can combine information from multiple related tables.

However, real-world database applications often require summary information rather than individual records.

For example:

- What is the total income recorded?
- What is the average income?
- Which income category has the highest income?
- How many income records belong to each category?
- Which category has a total income greater than ₹10,00,000?

SQL provides Aggregate Functions to perform such calculations.

The main aggregate functions covered in this laboratory are:

Function	Purpose

COUNT()	Counts records or values
SUM()	Calculates the total
AVG()	Calculates the average
MAX()	Finds the highest value
MIN()	Finds the lowest value

5. Database to be Used

Continue using the database created in the previous laboratories.

USE taxation_db;

The existing tables are:

- Taxpayer
- Income_Record
- Income_Category
- Financial_Year

No new tables are required for this laboratory.

6. Data Preparation

Aggregate functions and GROUP BY require multiple records to produce meaningful results.

Before starting the experiment, students must verify that sufficient data is available.

Execute:

SHOW TABLES;

Then display the existing records from:

```
SELECT * FROM Taxpayer;
SELECT * FROM Income_Category;
```

```
SELECT * FROM Financial_Year;  
SELECT * FROM Income_Record;
```

Students should ensure that the database contains multiple income records belonging to different income categories and financial years.

Recommended Data Conditions

Data	Recommended Minimum
Taxpayers	5–6
Income Categories	3–4
Financial Years	2
Income Records	10–15
Records in each category	At least 2
Records in each financial year	At least 2

If sufficient data is already available, students should continue with the experiment.

If additional records are required, students may insert additional records into `Income_Record` using valid existing `taxpayer_id`, `category_id`, and `year_id` values.

Students should not create duplicate categories or financial years.

7. Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single summary value.

7.1 COUNT()

Counts records or non-NULL values.

Syntax

```
SELECT COUNT(column_name)  
FROM table_name;
```

To count all rows:

```
SELECT COUNT(*)  
  FROM table_name;
```

COUNT() tells us how many records are available.

Example:

```
SELECT COUNT(*)  
  FROM Income_Record;
```

This displays the total number of income records.

7.2 SUM()

Calculates the total of a numeric column.

Syntax

```
SELECT SUM(column_name)  
  FROM table_name;
```

Example:

```
SELECT SUM(income_amount)  
  FROM Income_Record;
```

SUM() tells us the total income recorded in the table.

7.3 AVG()

Calculates the average value.

Syntax

```
SELECT AVG(column_name)  
  FROM table_name;
```

Example:

```
SELECT AVG(income_amount)
FROM Income_Record;
```

AVG() tells us the average income.

7.4 MAX()

Finds the highest value.

Syntax

```
SELECT MAX(column_name)
FROM table_name;
```

Example:

```
SELECT MAX(income_amount)
FROM Income_Record;
```

MAX() tells us the highest income recorded.

7.5 MIN()

Finds the lowest value.

Syntax

```
SELECT MIN(column_name)
FROM table_name;
```

Example:

```
SELECT MIN(income_amount)
FROM Income_Record;
```

MIN() tells us the lowest income recorded.

8. GROUP BY

GROUP BY is used to divide records into groups and perform an aggregate calculation for each group.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

GROUP BY means:

“Create separate groups based on this column and calculate the required value for each group.”

For example, if the income categories are Salary, Business, and Investment, GROUP BY can calculate the total income separately for each category.

Example

```
SELECT category_id, SUM(income_amount)
FROM Income_Record
GROUP BY category_id;
```

9. GROUP BY with Multiple Columns

GROUP BY can also be performed using more than one column.

Syntax

```
SELECT column1, column2, aggregate_function(column3)
FROM table_name
GROUP BY column1, column2;
```

Create groups using both columns and calculate the aggregate value for every combination.

Example:

```
SELECT category_id, year_id, SUM(income_amount)
FROM Income_Record
GROUP BY category_id, year_id;
```

This calculates the total income for each category in each financial year.

10. HAVING

HAVING is used to filter groups created using GROUP BY.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING aggregate_function(column_name) condition;
```

HAVING answers:

“After creating the groups, which groups should be displayed?”

Example:

```
SELECT category_id, SUM(income_amount)
FROM Income_Record
GROUP BY category_id
HAVING SUM(income_amount) > 1000000;
```

This displays only categories whose total income is greater than ₹10,00,000.

11. WHERE vs HAVING

WHERE	HAVING
Filters individual records	Filters groups
Applied before GROUP BY	Applied after GROUP BY

Used for row-level conditions

Example: `income_amount > 500000`

Commonly used with aggregate conditions

Example: `SUM(income_amount) > 1000000`

Easy way to remember

WHERE → Which rows?

HAVING → Which groups?

12. ORDER BY with Aggregate Functions

ORDER BY can be used to arrange the grouped results.

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
ORDER BY aggregate_function(column_name);
```

For descending order:

```
ORDER BY aggregate_function(column_name) DESC;
```

Example

```
SELECT category_id, SUM(income_amount)
FROM Income_Record
GROUP BY category_id
ORDER BY SUM(income_amount) DESC;
```

First calculate the total income for each category and then arrange the categories from highest total income to lowest.

13. Complete Query Structure

Students should remember the logical order of the clauses:

```
SELECT  
  FROM  
  WHERE  
  GROUP BY  
  HAVING  
  ORDER BY;
```

General Syntax

```
SELECT column_name, aggregate_function(column_name)  
  FROM table_name  
  WHERE condition  
  GROUP BY column_name  
  HAVING aggregate_function(column_name) condition  
  ORDER BY aggregate_function(column_name) DESC;
```

14. Aggregate Functions with JOIN

Since students have already learned JOIN operations in Task 4, aggregate functions can now be combined with JOINS.

For example, instead of displaying a category ID, we can display the actual category name.

```
SELECT C.category_name, SUM(I.income_amount)  
  FROM Income_Record I  
  JOIN Income_Category C  
  ON I.category_id = C.category_id  
  GROUP BY C.category_name;
```

JOIN gets the meaningful category name, while GROUP BY and SUM() calculate the total income for each category.

15. Laboratory Activities

Part A – Verify Previous Database

Before starting the experiment:

USE taxation_db;

Display all tables:

SHOW TABLES;

Verify that the following tables are available:

- Taxpayer
- Income_Category
- Financial_Year
- Income_Record

Display the existing data and ensure sufficient records are available.

Part B – Aggregate Functions

Level 1 – Understanding

Task 1: Display the total number of income records.

Hint: Use COUNT().

Task 2: Display the total income amount recorded in the database.

Hint: Use SUM().

Task 3: Display the average income amount.

Hint: Use AVG().

Task 4: Display the highest income amount recorded.

Hint: Use MAX().

Task 5: Display the lowest income amount recorded.

Hint: Use MIN().

Level 2 – Application

Task 1: Display the number of income records for each income category.

Hint: Use COUNT() with GROUP BY.

Task 2: Display the total income for each income category.

Hint: Use SUM() with GROUP BY.

Task 3: Display the average income for each income category.

Hint: Use AVG() with GROUP BY.

Task 4: Display the highest income recorded in each income category.

Hint: Use MAX() with GROUP BY.

Task 5: Display the lowest income recorded in each income category.

Hint: Use MIN() with GROUP BY.

Task 6: Display the total income for each financial year.

Hint: Group the records using year_id.

Task 7: Display the number of income records for each financial year.

Hint: Use COUNT() with GROUP BY.

Task 8: Display the total income for each income category in each financial year.

Hint: Use GROUP BY with category_id and year_id.

Level 3 – Medium to Advanced

Task 1: Display only those income categories whose total income is greater than ₹10,00,000.

Hint: Use GROUP BY and HAVING.

Task 2: Display income categories whose average income is greater than ₹5,00,000.

Hint: Use AVG() with HAVING.

Task 3: Display financial years having more than three income records.

Hint: Use COUNT() with HAVING.

Task 4: Display income categories in descending order of total income.

Hint: Use SUM() and ORDER BY ... DESC.

Task 5: Display income categories whose total income is greater than ₹10,00,000 and arrange them from highest total income to lowest.

Hint: Combine GROUP BY, HAVING, and ORDER BY.

Task 6: Display the total income and average income for each income category.

Hint: Use SUM() and AVG() together with GROUP BY.

Task 7: Display the category and financial year combination having the highest total income.

Hint: Use GROUP BY with two columns and ORDER BY.

Task 8: Display the number of taxpayers who have income records in each financial year.

Hint: Use JOIN, COUNT(), and GROUP BY.

16. Real-World Taxation Analysis

Task 1: Identify the income category that generates the highest total income.

Hint: Use SUM(), GROUP BY, and ORDER BY.

Task 2: Identify the financial year having the highest total recorded income.

Hint: Use SUM(), GROUP BY, and ORDER BY.

Task 3: Identify the income category having the highest average income.

Hint: Use AVG(), GROUP BY, and ORDER BY.

Task 4: Display income categories having more than two income records.

Hint: Use COUNT() with HAVING.

Task 5: Display financial years having total income greater than ₹10,00,000.

Hint: Use SUM(), GROUP BY, and HAVING.

Task 6: Generate a summary report containing Income Category, Number of Records, Total Income, Average Income, Highest Income, and Lowest Income.

Hint: Use multiple aggregate functions with GROUP BY.

17. Important Concept

Students should understand the following progression:

Individual Records



Aggregate Functions



GROUP BY



HAVING



ORDER BY



JOIN + Aggregate Functions



Summary Reports

The next laboratory will build on these concepts by introducing **Subqueries / Nested Queries**, where aggregate functions such as AVG(), MAX(), and MIN() can be used inside another SQL query.

18. Time-wise Laboratory Plan

Time	Activity
0–15 min	Verify previous database and existing data
15–30 min	Data preparation / insert additional records if required
30–50 min	Practice aggregate functions
50–75 min	Practice GROUP BY
75–100 min	Practice HAVING
100–115 min	Practice ORDER BY with aggregate results
115–140 min	Practice JOIN + Aggregate + GROUP BY
140–160 min	Execute Level 1, Level 2 and Level 3 tasks
160–180 min	GitHub submission, faculty verification and discussion

19. GitHub Submission

Students should create the following folder structure:

StudentID-repo

└─ Week-5-Lab

 └─ week5_aggregate_functions.sql

Verify Tables -

Taxpayer

taxpayer_id	taxpayer_name	pan_number	occupation
101	Arjun Kumar	ABCPA101A	Software Engineer
102	Priya Sharma	BCDPS102B	Business Owner
103	Ravi Teja	CDEPR103C	Doctor
104	Sneha Reddy	DEFPS104D	Teacher
105	Kiran Rao	EFGPK105E	Business Owner
106	Anjali Devi	FGHPA106F	Software Engineer

Income_Category

category_id	category_name	description
1	Salary	Income received from employment
2	Business	Income earned from business activities
3	Investment	Income earned from investments
4	Other Income	Income from other sources

Financial_Year

year_id	financial_year	start_date	end_date
1	2024-2025	2024-04-01	2025-03-31
2	2025-2026	2025-04-01	2026-03-31

Income_Record

income_id	taxpayer_id	category_id	year_id	income_source	income_amount

1001	101	1	1	ABC Technologies	600000
1002	102	2	1	Sharma Traders	900000
1003	103	1	1	City Hospital	750000
1004	104	1	1	Government School	500000
1005	105	2	1	Rao Enterprises	1200000
1006	106	3	1	Mutual Funds	300000
1007	101	1	2	ABC Technologies	700000
1008	102	2	2	Sharma Traders	1100000
1009	103	1	2	City Hospital	850000
1010	104	4	2	Rental Income	400000
1011	105	2	2	Rao Enterprises	1400000
1012	106	3	2	Mutual Funds	450000
1013	101	3	2	Stock Investment	250000
1014	104	4	1	Rental Income	350000